

00-2025 风格零基础期末总复习资料

目录

1	编译原理 2025 风格零基础期末突击总复习资料	2
---	--------------------------	---

1 编译原理 2025 风格零基础期末突击总复习资料

1.1 使用方法

这份资料按 2025 年期末卷重排复习重点。2025 卷的前两题延续旧卷，后四题明显转向后端和程序分析。复习时按下面顺序：

词法分析 -> LL 语法分析 -> 指令调度 -> 支配问题 -> 符号执行 -> Andersen 指针分析

旧卷中的 LR、ANTLR、SDD、TAC 仍保留为加练题，优先级低于 2025 主线。

1.2 一、2025 卷题型蓝图

1. 词法分析：正则表达式 -> Thompson NFA -> 子集构造 DFA -> DFA 最小化。
2. LL 语法分析：消除左递归 -> FIRST -> FOLLOW -> 预测分析表 -> LL(1) 判断。
3. 指令调度：目的、RAW/WAR/WAW、依赖图、寄存器重命名、重排指令。
4. 支配问题：Dom、PostDom、Dom Frontier、支配树或森林、给定 CFG 构造支配关系。
5. 符号执行：敏感性定义、CNF、Tseitin、同时可满足证明。
6. 指针别名：Andersen 约束、points-to 集合、图闭包推导。

1.3 二、词法分析模板

1.3.1 Thompson NFA

字符 a: start --a--> end
空串 epsilon: start --epsilon--> end
选择 A|B: 新 start epsilon 到 A/B 起点, A/B 终点 epsilon 到新 end
连接 AB: A 终点 epsilon 到 B 起点
闭包 A*: 新 start epsilon 到 A 起点和新 end, A 终点 epsilon 到 A 起点和新 end

答题步骤：

1. 按正则表达式括号从内到外拆分。
2. 每个子表达式套 Thompson 模板。
3. 给 NFA 状态编号。
4. 起点画入箭头，终点画双圈。

1.3.2 子集构造

epsilon-closure(S): 从 S 出发只走 epsilon 边到达的状态集合。
move(S,a): 从 S 出发读 a 到达的状态集合。
Start = epsilon-closure({NFA start}).
DFA 终态: NFA 状态集合中含 NFA 终态。

表格写法：

DFA 状态	NFA 状态集合	on a	on b	终态
A	{...}	B	C	否
B	{...}	B	D	是

1.3.3 DFA 最小化

$P0 = \{\text{终态集合}, \text{非终态集合}\}$

按每个输入字符的转移目标所在组拆分。

重复拆分直到分组不再变化。

每个分组变成一个最小 DFA 状态。

1.4 三、LL 语法分析模板

直接左递归：

$A \rightarrow A \text{ alpha} \mid \text{beta}$

消除：

$A \rightarrow \text{beta } A'$

$A' \rightarrow \text{alpha } A' \mid \text{epsilon}$

FIRST:

$\text{FIRST}(a) = \{a\}$

$\text{FIRST}(\text{epsilon}) = \{\text{epsilon}\}$

$\text{FIRST}(XY)$: 先看 $\text{FIRST}(X)$, 若 X 能推出 epsilon , 再看 $\text{FIRST}(Y)$ 。

FOLLOW:

开始符号 S : $\$$ in $\text{FOLLOW}(S)$

$A \rightarrow \text{alpha } B \text{ beta}$: $\text{FIRST}(\text{beta}) - \{\text{epsilon}\}$ 加入 $\text{FOLLOW}(B)$

$A \rightarrow \text{alpha } B$ 或 beta 能推出 epsilon : $\text{FOLLOW}(A)$ 加入 $\text{FOLLOW}(B)$

预测表:

对 $A \rightarrow \text{alpha}$:

$\text{FIRST}(\text{alpha}) - \{\text{epsilon}\}$ 中每个 a 填 $M[A, a]$

若 $\text{epsilon} \in \text{FIRST}(\text{alpha})$, $\text{FOLLOW}(A)$ 中每个 b 填 $M[A, b]$

表中无多重入口, 则是 LL(1)。

2025 原题模板:

$A \rightarrow B C$

$B \rightarrow B P \mid r$

$C \rightarrow t \mid u$

消除左递归:

```
A -> B C
B -> r B'
B' -> P B' | epsilon
C -> t | u
```

FIRST 与 FOLLOW:

```
FIRST(A)={r}
FIRST(B)={r}
FIRST(B')={P,epsilon}
FIRST(C)={t,u}
FOLLOW(A)={$}
FOLLOW(B)={t,u}
FOLLOW(B')={t,u}
FOLLOW(C)={$}
```

预测表:

```
M[A,r] = A -> B C
M[B,r] = B -> r B'
M[B',P] = B' -> P B'
M[B',t] = B' -> epsilon
M[B',u] = B' -> epsilon
M[C,t] = C -> t
M[C,u] = C -> u
```

1.5 四、指令调度模板

目的:

在不改变程序语义的前提下重排指令，减少流水线停顿，提高并行执行效率。

三类依赖:

True dependence / RAW: i 写 x, j 读 x, i 必须在 j 前。
 Anti dependence / WAR: i 读 x, j 写 x, j 不能提前破坏 i 读到的旧值。
 Output dependence / WAW: i 写 x, j 写 x, 二者顺序影响最终写入值。

寄存器重命名:

能消除 WAR 和 WAW。
 不能消除 RAW。

答题步骤:

1. 每条指令一个节点。
2. RAW 画实线边。
3. WAR 和 WAW 画虚线边。
4. 对写后写、读后写冲突做寄存器重命名。
5. 对剩余依赖图给一个拓扑序。

1.6 五、支配问题模板

定义：

Dom(n): 从入口到 **n** 的每条路径都经过的节点集合。
PostDom(n): 从 **n** 到出口的每条路径都经过的节点集合。
idom(n): **n** 的最近严格支配者。
Dom Frontier(n): **n** 支配某个前驱，但不严格支配该节点的位置。

计算 Dom:

```
Dom(entry) = {entry}
其他节点 Dom 初值为全集
Dom(n) = {n} union intersection(Dom(p) for p in pred(n))
迭代到不再变化
```

支配关系图是森林：

除入口外，每个可达节点有唯一 **idom**。
idom 严格支配该节点。
 严格支配关系无环。
 因此每个节点至多一个父节点，且无环，构成树；多入口时构成森林。

1.7 六、符号执行与 SAT 模板

敏感性：

Path-sensitive: 区分不同执行路径。
Flow-sensitive: 区分程序点和语句顺序。
Context-sensitive: 区分函数调用上下文。
Field-sensitive: 区分对象字段。

CNF:

CNF 是子句的合取，每个子句是文字的析取。
 (a or not b) and (c or d)

Tseitin:

1. 给每个子表达式引入新变量。
2. 为新变量和子表达式的语义关系写 CNF。
3. 加入根变量为真的子句。
4. 得到与原式同时可满足的 CNF。

常用子句:

```

x <-> (a and b)
(not x or a) and (not x or b) and (x or not a or not b)

x <-> (a or b)
(x or not a) and (x or not b) and (not x or a or b)

x <-> not a
(not x or not a) and (x or a)

```

同时可满足证明模板：

```

F1 = (a0 or ... or an or c) and (b0 or ... or bm or not c)
F2 = (a0 or ... or an or b0 or ... or bm)

```

F1 $\rightarrow$ F2:

若 $c=false$ ，则第一子句要求某个 $ai=true$ 。

若 $c=true$ ，则第二子句要求某个 $bj=true$ 。

因此 $F2=true$ 。

F2 $\rightarrow$ F1:

若某个 $ai=true$ ，令 $c=false$ 。

若某个 $bj=true$ ，令 $c=true$ 。

因此 $F1=true$ 。

1.8 七、Andersen 指针分析模板

points-to 集合：

$pts(p) = p$ 能指向的对象集合

四类语句：

```

y = &x: x in pts(y)
y = x: pts(x) subset pts(y)
*y = x: for v in pts(y), pts(x) subset pts(v)
y = *x: for v in pts(x), pts(v) subset pts(y)

```

闭包步骤：

1. 先处理取地址语句，得到初始 **points-to** 集。
2. 处理复制语句，传播集合包含关系。
3. 处理 **store** 语句，把右侧集合写入左侧指向对象的集合。
4. 处理 **load** 语句，把被指对象的集合读入左侧变量。
5. 有新增元素就继续传播，直到不再变化。

1.9 八、旧卷加练题

ANTLR:

先写运算符优先级和结合性，再按优先级给语法树。

例如 ! 最高, ^ 右结合, + 最低且左结合:

$1 + 2^3^4 + 5!! = (1 + (2^{(3^4)})) + ((5!)!)$

SDD:

先定义属性，再逐产生式写语义规则。

二进制小数: L.val, L.len, B.val, S.val。

TAC 回填:

B.truelist, B.falselist, S.nextlist

makelist, merge, backpatch

while 的假链就是循环出口，真链进入循环体。

LR:

项目: [A -> alpha . beta, a]

closure 和 goto 按模板写。

归约项 [A -> alpha ., a] 只在 a 列填 reduce。

1.10 九、最后背诵清单

Thompson 五模板

子集构造三操作: epsilon-closure、move、终态判断

DFA 最小化初分组和拆分

左递归消除公式

FIRST/FOLLOW/预测表规则

RAW/WAR/WAW 定义

寄存器重命名能消除 WAR/WAW

Dom/PostDom/Dom Frontier 定义

支配树是森林的证明

Path/Flow/Context/Field sensitivity

CNF 与 Tseitin 三类子句

Andersen 四类约束